Fault-tolerant control for UAV based on deep reinforcement learning under single rotor failure

Weiyi Cheng
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
chengwy5@mail2.sysu.edu.cn

Han Li
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
lihan68@mail2.sysu.edu.cn

Zhiwei Hou*
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
houzhw5@mail.sysu.edu.cn

2024 36th Chinese Control and Decision Conference (CCDC) | 979-8-3503-8778-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/CCDC62350.2024.10588308

***Abstract*—In order to achieve high-speed flight of a quadrotor UAV when one single rotor completely fails, a fault-tolerant control method based on deep reinforcement learning was designed. Since our policy is a low-level control, neural networks are used in reinforcement learning to directly map states to motor action values. The learning algorithm is developed based on the proximal policy optimization algorithm. By introducing PID lift correction into the actor-critic structure, both control accuracy and robustness are significantly improved. The proposed method has been tested in a flight simulator, and the results show that the reinforcement learning strategy is highly robust to model uncertainty and external interference.**

Keywords—Quadcopter UAV, deep reinforcement learning, fault-tolerant control

I. INTRODUCTION

Thanks to the swift advancements in control science, integrated circuits, and computer technology, drones have achieved remarkable progress in recent years. Unmanned Aerial Vehicles (UAVs) offer the advantages of excellent maneuverability, cost-effectiveness, and a notable load-bearing capacity. They play a pivotal role in various domains, encompassing military applications as well as civilian sectors, including disaster relief, field-based scientific research, collaborative operations, etc. Nonetheless, owing to its underactuated nature and the highly nonlinear, tightly interrelated dynamics, the design of a controller for a quadcopter UAV to attain exceptional control performance is an exceedingly challenging task. Moreover, while a quadrotor drone is in flight, it may experience unforeseen external disturbances, contend with environmental complexities, and grapple with its own reliability factors, all of which can potentially lead to sensor or actuator malfunctions, ultimately posing significant safety risks. Therefore, it is necessary to design a fault-tolerant controller for a quadrotor UAV when one rotor completely fails to maintain good dynamic performance of the system as much as possible.

Many previous works used traditional control methods to achieve fault-tolerant control of UAVs. For example, Sun *et al.* used incremental dynamic inverse control combined with PID algorithm to achieve fault-tolerant position control of UAVs using only airborne sensors, which can track a Series fixed point and withstand motion blur [1]. Non-cascade nonlinear MPC control is used in [2], which considers the complete UAV model and the thrust constraints of each rotor, and can recover from a single rotor failure scenario in any attitude. Cascade LQR is used to propose an algorithm that allows a quadrotor UAV to stabilize around the target point even after losing one, two, or three rotors [3]. Algorithms such as LPV, SQL, back-stepping method, H_∞ and non-singular terminal sliding mode are respectively used to solve the problem of complete failure of a single rotor fault-tolerant control issues [4]-[8].

In addition, reinforcement learning can also be used to solve the attitude control problem of UAVs. Hwangbo *et al.* proposed a model-free neural network algorithm that can track fixed points and recover from harmful attitude at a control frequency of 7 microseconds per step, and transplanted it to real machine [9]. Lambert *et al.* proposed a model-based deep reinforcement learning UAV attitude control algorithm that can converge quickly and operate at a control frequency no higher than 50hz [10]. J. Yoo *et al.* proposed an algorithm that combines PD or LQR with model-based deep reinforcement learning, which can achieve faster convergence speed and better control performance [11]. These papers all used the proximal policy optimization algorithm or improved it to achieve the position control of the UAV when the rotors are intact [12]-[16]. Among them, the integral compensator is added to the observation matrix to eliminate the steady-state error [15]. Xue W *et al.* used Monte Carlo approximation to replace the KL divergence to improve the control accuracy [16]. C. Xiao *et al.* proposed using the SAC algorithm to realize a quadcopter UAV passing through an inclined narrow gap [17]. Kooi J E *et al.* used the PPO algorithm to implement a simulation experiment of a drone hovering and landing on a slope, and directly transplanted it to the actual machine [18].

In the field of fault-tolerant control, Sharma P *et al.* used the SAC algorithm to achieve stable hovering, landing and tracking of a UAV when one rotor completely failed, but there was a certain lag in control performance [19]. Arasanipalai R *et al.* proposed a method that combines deep reinforcement learning algorithm with PD control to achieve control under complete failure of one or two rotors, and uses position smoothing method to eliminate steady-state errors during the training process [20]. Bello R proposed a hierarchical reinforcement learning algorithm to deal with a complete rotor failure scenario, where the outer and inner circles are trained separately and the former is used to provide the desired angular velocity to the latter [21]. Medium supervised learning is combined with low-level PID control to achieve important stability guarantees, which can cope with rotor failure, wind disturbance and severe position and attitude noise [22]. F. Fei *et al.* proposed a control algorithm based on reinforcement learning for failures or strikes of actuators and sensors, which can solve about 30% of rotor failures [23]. N. Bernini conducted robust control of reinforcement learning against motor failure and wind disturbance, and verified that the control performance of the direct training model under lossy conditions is better than that of the training model under lossless conditions [24].

*Corresponding author: Zhiwei Hou, e-mail: houzhw5@mail.sysu.edu.cn
This work is supported by National Natural Science Foundation of China under Grant 62203481

The main contributions of this paper are as follows. We use a model-free deep reinforcement learning algorithm which is proximal policy optimization algorithm to achieve fault-tolerant control of UAVs. It does not need to learn complete state transition probabilities function like model-based deep reinforcement learning algorithms. The PID algorithm was used to eliminate the steady-state error on the Z-axis, and the remaining three rotors were finally used to achieve reliable and stable flight of the UAV, including hovering and resistance to wind disturbance.

The subsequent structure of this article is as follows. Section II introduces the dynamic model of quadrotors. The section III describes the reinforcement learning algorithm, network structure and reward function design. Section IV validates the controller in simulator and analysis the results.

II. PRELIMINARIES

We consider the quadrotor as a rigid body with four rotors positioned at the corners of a rectangular frame and in the × configuration (Fig. 1) which is the most widely used. This frame is aligned parallel to the X-Y plane of the body's coordinate system, as are the rotors. Two rotors rotate clockwise and the other two counterclockwise to balance the body and create yaw movement as needed. The dynamic model of the quadcopter UAV is as follows:

$$\begin{cases} m\ddot{\xi} = m\mathbf{g} + R_{IB} f^B \\ \tau = I_v^B \dot{\omega}^B + \omega^B \times I_v^B \omega^B \\ \dot{R}_{IB} = \omega^B{}_{\times} R_{IB} \end{cases} \tag{1}$$

where represents the total load mass of the quadrotor, $\xi = [x, y, z]^T$ is the position of the aircraft center of mass in the inertial frame, $\mathbf{g} = [0, 0, -9.81]^T$ represents the gravity vector, $R_{IB} \in SO(3)$ (2) is the rotation matrix of the UAV from body frame to the inertial frame, θ_q , ϕ_q and ψ_q respectively represents roll angle, pitch angle and yaw angle, $f^B = [0, 0, T_{total}]^T$ is the total force vector in the body frame, T_{total} is the collective thrust produced by the four rotors , $\tau = [\tau_x, \tau_y, \tau_z]^T$ is the control torques in the body frame, $\omega^B = [\omega_x, \omega_y, \omega_z]^T$ is the angular velocity in the body frame, $I_v^B \in R^{3\times3}$ represents the inertia tensor, $\omega_{\times}^B \in R^{3\times3}$ is a skew-symmetric matrix associated with ω^B rotated to the inertial frame.

$$R_{IB} = \begin{bmatrix} \cos\psi_q \cos\phi_q & \cos\psi_q \sin\theta_q \sin\phi_q & \cos\psi_q \sin\theta_q \cos\phi_q + \sin\psi_q \sin\phi_q \\ \sin\psi_q \cos\theta_q & \sin\psi_q \sin\theta_q \sin\phi_q & \sin\psi_q \sin\theta_q \cos\phi_q - \sin\psi_q \cos\phi_q \\ -\sin\theta_q & \cos\theta_q \sin\phi_q & \cos\theta_q \cos\phi_q \end{bmatrix} \tag{2}$$

When a rotor completely fails, in the simulator we set the failed rotor to no longer produce any lift. In this case, the total lift and each axial moment are greatly different from those in the case where the four rotors are intact. Because of the under-actuated nature of the quadrotor UAV, it's important to note that the yaw angle is uncontrollable under these conditions. The failure of a rotor can lead to significant alterations in the UAV's dynamic model. In the previously mentioned article, the conventional controller design requires the incorporation of the UAV's precise model, which in turn diminishes its overall robustness. Therefore, we employ a model-free controller. A deep reinforcement learning algorithm proposes a robust fault-tolerant controller to deal with single rotor failure. We assume that reasonably accurate estimates of the quadrotor's position, velocity, orientation, and angular velocity are available and the UAV already has the fault detection capability proposed in [25]. Once the UAV detects the failure of a rotor, the reinforcement learning controller will immediately take over the low-level controller.

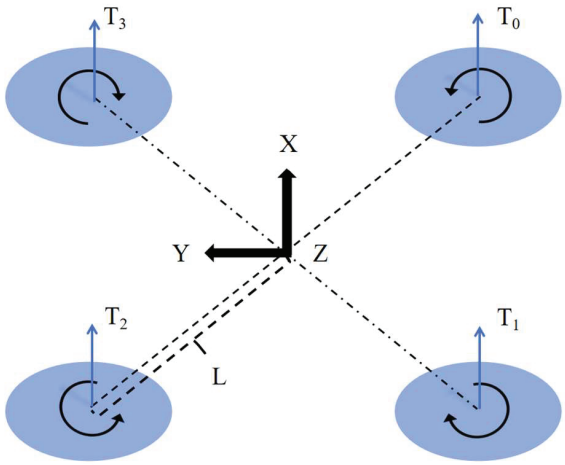

Fig. 1. Top-down view of our generalized quadrotor model. The model always assumes the × configuration, with the front and left pointing at the positive X and Y directions respectively. The rotors 0, 1, 2, 3 at the ends of each arm of length L, generate upward forces T_0, T_1, T_2 and T_3.

III. METHODOLOGY

This section primarily focuses on the application of a reinforcement learning algorithm, specifically, the Proximal Policy Optimization (PPO) algorithm, for addressing the control problem of quadrotor UAVs in the event of a single-rotor failure. Compared with DDPG, SAC, and TRPO, PPO is simpler and does not have non-convergence problems. The overarching control objective of this project is to enable the quadcopter UAV to rapidly, steadily, and precisely reach the designated target point in the event of a failure, minimizing deviations from the target trajectory and ensuring strong dynamic performance.

A. Problem Formulation

First, define the input of the reinforcement learning neural network as a series of states $s_t = (e_p, v, R_{flat}, \omega^B)$ of the UAV, where $e_p = \xi_{goal} - \xi$ is the position error of the UAV in the inertial frame, $\xi_{goal} = [x_{goal}, y_{goal}, z_{goal}]^T$ is the goal position in the inertial frame, $v = [v_x, v_y, v_z]^T$ is the linear velocity of the UAV in the inertial frame. In order to avoid the ambiguity of a pair of quaternions with different signs representing the same attitude, $R_{flat} \in R^9$ is the flattened form of quadrotor's rotation matrix R_{IB} .

In order to stabilize the UAV when the single rotor fails, it is necessary to align the desired thrust direction with the main axis as much as possible, and it is assumed that the yaw angular velocity remains stable after a certain duration of

flight following the failure [1]. Hence, in the design of the reward function, it is imperative to take into account the roll and pitch of the UAV while also considering the angular velocity of its corners, the following is (3) the designed reward function:

$$
\begin{aligned}
reward &= -dt(r_{pos} + r_{alt} + r_{act} \\
&+ r_{vel} + r_{orient} + r_{spin} + r_{crash}) \\
r_{pos} &= rew_{pos}(c_1 \left\| \xi_{goal} - \xi \right\|_2 + \\
&\tanh(c_2 \left\| \xi_{goal} - \xi \right\|_2)) \\
r_{alt} &= rew_{alt}\left(\exp\left(z_{goal} - z\right) - c_3\right) \\
r_{act} &= rew_{act}\sqrt{\sum_{i=0}^{3}\left(a_t^i - a_{t-1}^i\right)^2} \\
r_{vel} &= rew_{vel}\left\| v \right\|_2 \\
r_{spin} &= \begin{cases} rew_{spin}, \ if \ \left|\omega_x\right| > 10 \ or \ \left|\omega_y\right| > 10 \\ 0, \ if \ \left|\omega_x\right| \le 10 \ and \ \left|\omega_y\right| \le 10 \end{cases} \\
r_{crash} &= \begin{cases} rew_{crash}, \ if \ crashed = 1 \\ 0, \ if \ crashed = 0 \end{cases}
\end{aligned}
\tag{3}
$$

where rew_{pos}, rew_{alt}, rew_{act}, rew_{vel}, rew_{spin}, rew_{crash} and $c_{1,2,3}$ are constants. r_{pos} represents a positional constraint term. To maximize the drone's proximity to the target point, we incorporate a combination of linear and tanh terms, with 'tanh' representing the hyperbolic tangent function in our approach. Since the tanh term rises rapidly when it is close to the zero point, it can better stimulate agent converges to the target trajectory. r_{alt} is an incentive for height, because when one rotor fails, the drone can easily not learn the correct control distribution to take off and exp is exponential function. r_{act} is a limit on the rapid change of the network output action to prevent the rapid change of action from causing instability to the UAV attitude during the training process, $a_t = \left[a_t^0, a_t^1, a_t^2, a_t^3\right]^T$ and $a_{t-1} \in R^4$ respectively represent the action values of each motor at the current time step and the previous time step. r_{vel} and r_{crash} are penalties for the drone's speed and crash respectively, *crashed* is a Boolean value that is 'true' when the drone crashes. r_{spin} represents the penalty applied when the absolute value of the UAV's angular velocity around the X-axis or Y-axis exceeds 10 rad/s. Our objective is to attain an attitude that maintains UAV stability within this range under SRF (Single Rotor Failure) conditions. The total reward is the sum of the reward components and multiplied by $-dt$, where dt is dynamic integration period.

B. Network structure

The policy network is used to map the observed state into an action value. To ensure the policy distribution maintains a mean of zero and unit variance, the actor network θ output corresponds to a_t, while the actual lift produced by the motor is represented by $\hat{f} = \left[\hat{f}_0, \hat{f}_1, \hat{f}_2, \hat{f}_3\right]^T$, $\hat{f}_i \in [0,1]$. The mapping relationship between a_t^i and $\hat{f}_i$ is denoted as $\hat{f}_i = (a_t^i + 1)/2$. When $\hat{f}_i = 0$ means no motor lift, and when $\hat{f}_i = 1$ means the full lift which is related to the thrust-to-weight coefficient r_{t2w} of the drone itself. The final motor thrusts are computed as $\mathbf{T_{vector}} = \hat{f} f_{\max}$, where $\mathbf{T_{vector}} = \left[T_0, T_1, T_2, T_3\right]^T$, $f_{\max} = 0.25 mg r_{t2w}$, g is the gravity constant. The critic network ϕ serves the purpose of estimating the state's value, which represents the reward associated with that state. Both the actor network and the critic network contain two hidden layers. The output layer of the former is four neurons, and the output layer of the latter is one neuron. Fig. 2 and Fig. 3 represent the schematic diagrams of the actor network and the critic network respectively. TABLE I shows the parameter settings of the neural networks.

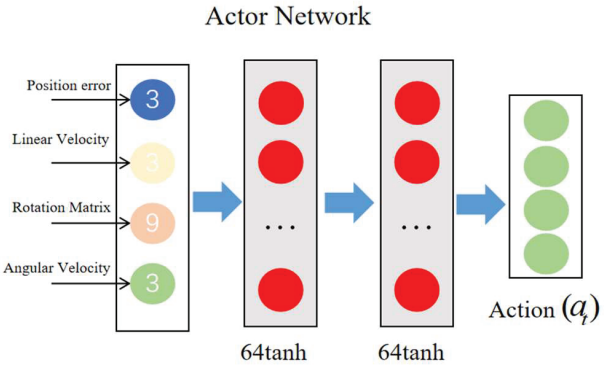

Fig. 2. Actor network structure

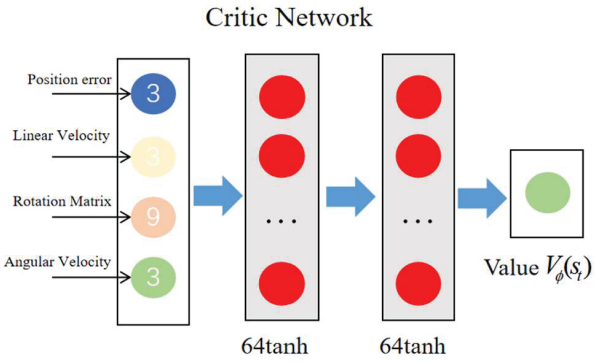

Fig. 3. Critic network structure

TABLE I Neural network parameters

Hyper-parameter	*Actor*	*Critic*
Hidden Layer1 size	64	64
Hidden Layer1 Activation	tanh	tanh
Hidden Layer2 size	64	64
Hidden Layer2 Activation	tanh	tanh
Learning rate	1e-3	1e-3

C. Training algorithm

The actor network takes an action in the current state s_t of each step. The simulator executes a step to obtain the next state s_{t+1} and a reward value r_t until the end of this trajectory. The advantage function $A_t(s_t, a_t)$ of the PPO algorithm is calculated as follows:

$$A_t(s_t,a_t)=Q_\theta(s_t,a_t)-V_\phi(s_t) \tag{4}$$

$$Q_\theta(s_t,a_t)=E_t\left[\sum_{l=0}^{\infty}\gamma^l r_{t+l}\mid s_t,a_t\right] \tag{5}$$

where $Q_\theta(s_t,a_t)$ is action value function, $V_\phi(s_t)$ is state value function, E_t represents the expected value, l is the length of time steps after t and γ is discount factor. Since the proximal policy optimization algorithm is a heterogeneous strategy, the learning agent and the agent interacting with the environment are not the same. In order to avoid the excessive time cost of resampling after updating the parameters once, we use the concept of importance sampling to solve this problem. The pruning algorithm in proximal policy optimization is selected to avoid the calculation of KL divergence. The objective function $J(\theta)$ as follows, where $\text{clip}(r_t(\theta),1-\varepsilon,1+\varepsilon)$ is a clipping function that limits $r_t(\theta)$ to the range $[1-\varepsilon,1+\varepsilon]$, min function is used to find the smallest element in the given set. A key advantage of the objective function is that it allows control over the magnitude of policy updates, resulting in more stable convergence to a better policy:

$$J(\theta)=E_t[\min(r_t(\theta)A_t,\text{clip}(r_t(\theta),1-\varepsilon,1+\varepsilon)A_t)] \tag{6}$$

where $r_t(\theta)=\pi_\theta(a_t\mid s_t)/\pi_{\theta old}(a_t\mid s_t)$, π_θ and $\pi_{\theta old}$ represent the new and old strategies respectively. The PPO algorithm is as follows:

Algorithm 1 Proximal Policy Optimization Algorithm

for iteration=1, 2, … **do**
  for actor=1, 2, …, N **do**
    Run policy $\pi_{\theta old}$ in the environment for T timesteps
    Compute advantage functions $A_1(s_1,a_1),...,A_T(s_T,a_T)$
  end for
  Optimize surrogate $J(\theta)$, with K epochs and minibatch size $M \le NT$
  $\theta_{old}\leftarrow\theta$
end for

D. Complete algorithm

There are competing paradigms of bias between variance in deep reinforcement learning. It can be observed that there is a significant steady-state error problem on the Z-axis when the PPO algorithm is applied to UAV control problems. It is difficult to reduce this error by increasing the number of training rounds or using other reward functions. An important reason for this problem may be that the action value function is not accurate. We combined the algorithm proposed in [1] to correct the UAV lift to reduce the steady-state error. First get the desired acceleration a_{des} based on a PID controller, where k_p is proportional gain, k_d is derivative gain, k_i is integral gain:

$$a_{des}=k_p(\xi_{goal}-\xi)+k_d(\dot{\xi}_{goal}-v)+k_i\int(\xi_{goal}-\xi)dt,\quad (k_p,k_d,k_i>0) \tag{7}$$

The modified thrust T_{mod} can be obtained from the expected vertical acceleration $a_{des,z}$ and (1):

$$T_{mod}=ma_{des,z}/\cos\phi_q\cos\theta_q \tag{8}$$

The output motor value of the reinforcement learning network is converted into lift and torque, and then the modified lift output by the PID controller is added to the former. The simulator then applies the total lift and torque to the drone, which can eliminate the steady-state error of the Z-axis. The UAV control structure diagram is shown in Fig. 4, and the PID parameters are shown in TABLE II, lift and torque are calculated as follows:

$$\left[T_{total},\tau^T\right]^T=G\mathbf{T}_{\mathbf{vector}} \tag{9}$$

$$G=\begin{bmatrix}1 & 1 & 1 & 1\\ -\sqrt{2}\mathrm{L}\kappa_0/2 & -\sqrt{2}\mathrm{L}\kappa_0/2 & \sqrt{2}\mathrm{L}\kappa_0/2 & \sqrt{2}\mathrm{L}\kappa_0/2\\ -\sqrt{2}\mathrm{L}\kappa_0/2 & \sqrt{2}\mathrm{L}\kappa_0/2 & \sqrt{2}\mathrm{L}\kappa_0/2 & -\sqrt{2}\mathrm{L}\kappa_0/2\\ -\tau_0 & \tau_0 & -\tau_0 & \tau_0\end{bmatrix} \tag{10}$$

where $G\in R^{4\times4}$ is the control effective matrix projecting individual thrust of each rotor to the collective thrust and torques, where κ_0 and τ_0 are force and torque coefficients respectively.

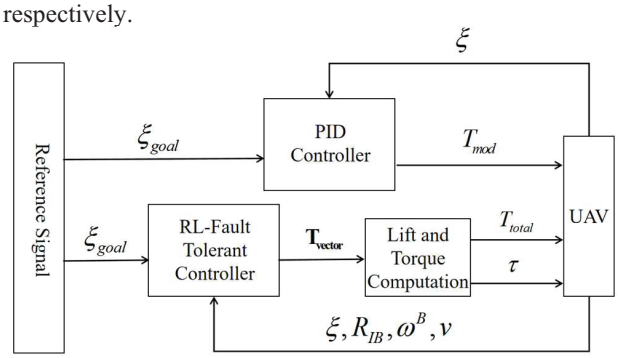

Fig. 4. UAV control structure diagram

TABLE II PID parameters

Hyper-parameter	X	Y	Z
k_p	3	3	12
k_d	1.4	1.4	8
k_i	0	0	10

IV. Simulation and Analysis

This section is mainly about simulation and analysis. In the subsequent construction of the simulation environment, we adopted Garage which is an open source and dynamically maintained TensorFlow-based reinforcement learning framework.

A. Simulator settings

In order to train the network, we collected forty trajectories with a duration of 7s in each round. In the simulation, the control frequency of the strategy is 100hz, but the dynamic integration frequency is 200hz, i.e. $dt=0.005$ms. The hyper-parameters of algorithm are $\gamma=0.99$, $\varepsilon=0.05$. During the strategy optimization process, the initial state of the UAV will be randomly sampled, the attitude is sampled at all $SO(3)$, the position is

sampled 2m near the target point, the maximum sampling value of the linear velocity is 1m/s , and the maximum sampling value of the angular velocity is 2πrad/s , and the target point is set at $[0,0,2]^T$ in the inertial frame.

We set up the broken rotor to not produce any lift and trained the controller for 6000 epochs on an Nvidia Geforce 2070 with 16 GB of memory. The parameters of reward function are showed in TABLE III, the parameters of the UAV model Crazeflie2.0 we used are shown in TABLE IV, and the visual simulation is shown in Fig. 5:

TABLE III Reward function parameters

rew_{pos}	6
rew_{alt}	2
rew_{act}	0.5
rew_{vel}	0.5
rew_{crash}	1
rew_{spin}	1
c_1	0.1
c_2	1
c_3	0.5

TABLE IV Quadrotor parameters

Variable(unit)	Value
$m(\mathrm{kg})$	0.028
L(m)	0.065
$r_{t2w}(\mathrm{kg/N})$	1.9
$I_v^B(\mathrm{kg{\cdot}m^2})$	$\begin{bmatrix} 1.3669e-5 & 0 & 0 \\ 0 & 1.4357e-5 & 0 \\ 0 & 0 & 2.6562e-5 \end{bmatrix}$

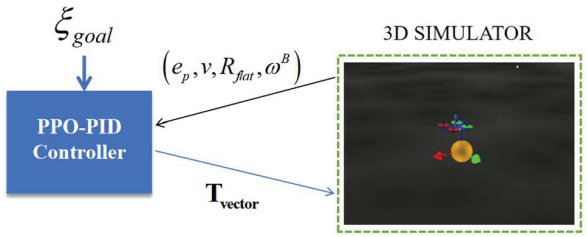

Fig. 5. 3D Visual Simulator

B. Hovering

In this experiment, the lift of the UAV's No. 3 rotor was set to 0, the initial position was $[0,0,0.05]^T$, and the target point was set at $[0,1,2]^T$. The position change of the UAV can be observed when the single rotor completely fails in Fig. 6. After 5 seconds, all three axes of the UAV converge to the target position. Fig. 7 is the action value output by the UAV neural network. Motor No. 3, when shut down, essentially provides no thrust output. The remaining three rotors provide torque and lift control. Among these, the two rotors opposing the speed of motor No. 3 are responsible for providing greater control output. Fig. 8 is the three-dimensional trajectory of the UAV.

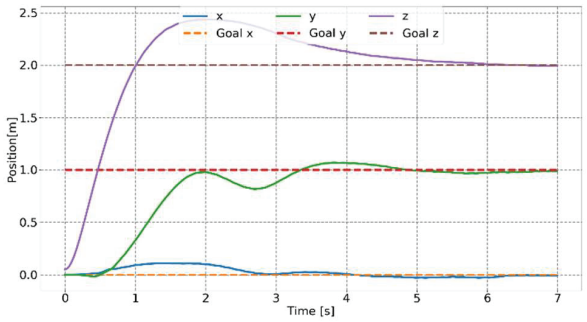

Fig. 6. Position of drone in fault-tolerant mode

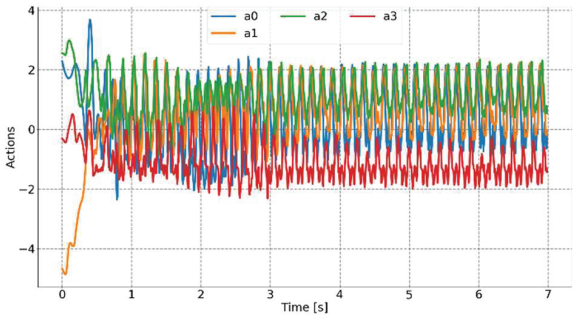

Fig. 7. Action value of UAV in fault-tolerant mode

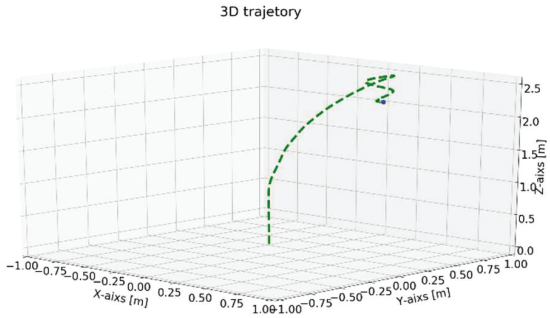

Fig. 8. Three-dimensional trajectory of UAV in fault-tolerant mode

C. Wind tolerance

Since wind disturbance does not have Markov properties, wind disturbance is not added to the learning process. In order to test the robustness of the neural network controller to wind disturbance, we added random direction winds with wind speeds up to 5m/s into the model and found that the UAV converged to the target except for a slight steady-state error on the Y-axis, the results show that the reinforcement learning fault-tolerant controller still has excellent control performance under wind disturbance. Fig. 9 - Fig. 11 respectively show the fault-tolerant control position, action value and three-dimensional trajectory of the UAV under wind disturbance.

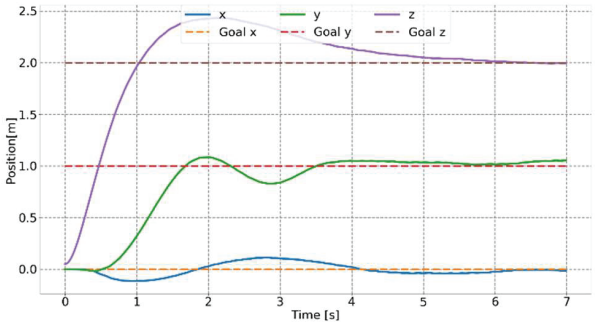

Fig. 9. Position of drone under the wind disturbance of SRF

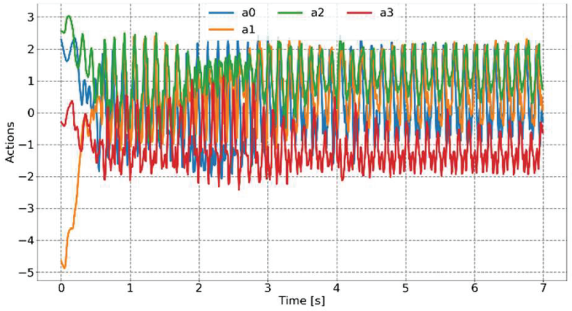

Fig. 10. Action value of drone under the wind disturbance of SRF

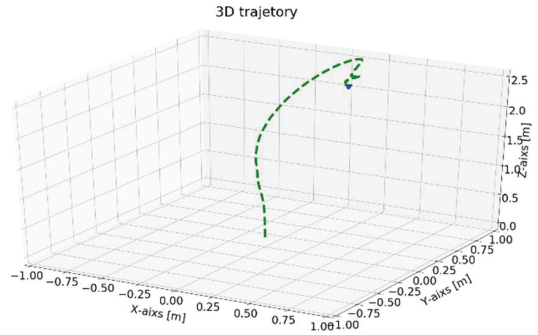

Fig. 11. Three-dimensional trajectory of UAV under the wind disturbance of SRF

CONCLUSION

In this article, we use the algorithm of proximal policy optimization to construct a fault-tolerant controller for a quadrotor UAV in the case of complete single-rotor failure. The reward function is used to design the stable interval of the body's angular velocity so that the algorithm can find the stable attitude in the failure situation, and the PID algorithm is used to eliminate the steady-state error. The proposed algorithm operates at a control frequency of 100Hz, offering excellent control performance and robustness. It effectively stabilizes hovering and can resist wind disturbances. In future work, the performance of the proposed controller needs to be tested on a real machine, and PID algorithm in the outer loop can be removed to make the reinforcement learning method simpler. Path planning, multi-agent clusters and the proposed algorithm can be combined to achieve higher levels of intelligent control.

REFERENCES

[1] Sun S, Cioffi G, De Visser C, et al. Autonomous quadrotor flight despite rotor failure with onboard vision sensors: Frames vs. events[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 580-587.

[2] Nan F, Sun S, Foehn P, et al. Nonlinear MPC for quadrotor fault-tolerant control[J]. IEEE Robotics and Automation Letters, 2022, 7(2): 5047-5054.

[3] Mueller M W, D'Andrea R. Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers[C]//2014 IEEE international conference on robotics and automation (ICRA). IEEE, 2014: 45-52.

[4] Stephan J, Schmitt L, Fichter W. Linear parameter-varying control for quadrotors in case of complete actuator loss[J]. Journal of Guidance, Control, and Dynamics, 2018, 41(10): 2232-2246.

[5] De Crousaz C, Farshidian F, Neunert M, et al. Unified motion control for dynamic quadrotor maneuvers demonstrated on slung load and rotor failure tasks[C]//2015 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2015: 2223-2229.

[6] V. Lippiello, F. Ruggiero and D. Serra, "Emergency landing for a quadrotor in case of a propeller failure: A backstepping approach," 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, Chicago, IL, USA, 2014, pp. 4782-4788, doi: 10.1109/IROS.2014.6943242.

[7] Lanzon A, Freddi A, Longhi S. Flight control of a quadrotor vehicle subsequent to a rotor failure[J]. Journal of Guidance, Control, and Dynamics, 2014, 37(2): 580-591.

[8] Hou Z, Lu P, Tu Z. Nonsingular terminal sliding mode control for a quadrotor UAV with a total rotor failure[J]. Aerospace Science and Technology, 2020, 98: 105716.

[9] Hwangbo J, Sa I, Siegwart R, et al. Control of a quadrotor with reinforcement learning[J]. IEEE Robotics and Automation Letters, 2017, 2(4): 2096-2103.

[10] Lambert N O, Drew D S, Yaconelli J, et al. Low-level control of a quadrotor with deep model-based reinforcement learning[J]. IEEE Robotics and Automation Letters, 2019, 4(4): 4224-4230.

[11] J. Yoo, D. Jang, H. J. Kim and K. H. Johansson, "Hybrid Reinforcement Learning Control for a Micro Quadrotor Flight," in IEEE Control Systems Letters, vol. 5, no. 2, pp. 505-510, April 2021, doi: 10.1109/LCSYS.2020.3001663.

[12] Molchanov, A., Chen, T., Hönig, W., Preiss, J.A., Ayanian, N., and Sukhatme, G.S. (2019). Sim-to-(Multi)-Real: Transfer of low-level robust control policies to multiple quadrotors. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 59–66. doi:10.1109/IROS40897.2019.8967695.

[13] Wang, Qiansong et al. “End-to-End Low-level Hovering Control of Quadrotor Based on Proximal Policy Optimization.” 2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT) (2022): 1300-1304.

[14] Pi C H, Hu K C, Cheng S, et al. Low-level autonomous control and tracking of quadrotor using reinforcement learning[J]. Control Engineering Practice, 2020, 95: 104222.

[15] Hu H, Wang Q. Proximal policy optimization with an integral compensator for quadrotor control[J]. Frontiers of Information Technology & Electronic Engineering, 2020, 21(5): 777-795.

[16] Xue W, Wu H, Ye H, et al. An Improved Proximal Policy Optimization Method for Low-Level Control of a Quadrotor[C]//Actuators. MDPI, 2022, 11(4): 105.

[17] C. Xiao, P. Lu and Q. He, "Flying Through a Narrow Gap Using End-to-End Deep Reinforcement Learning Augmented With Curriculum Learning and Sim2Real," in IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 5, pp. 2701-2708, May 2023, doi: 10.1109/TNNLS.2021.3107742.

[18] Kooi J E, Babuška R. Inclined quadrotor landing using deep reinforcement learning[C]//2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2021: 2361-2368.

[19] Sharma P, Poddar P, Sujit P B. A Model-free Deep Reinforcement Learning Approach To Maneuver A Quadrotor Despite Single Rotor Failure[J]. arXiv preprint arXiv:2109.10488, 2021.

[20] Arasanipalai R, Agrawal A, Ghose D. Mid-flight propeller failure detection and control of propeller-deficient quadcopter using reinforcement learning[J]. arXiv preprint arXiv:2002.11564, 2020.

[21] Bello R. Reinforcement Learning for Quadcopter Fault Tolerance: Analyzing Hierarchical and Online Deep Reinforcement Learning for Quadcopter Fault Tolerant Control[J]. 2021.

[22] Sohège Y, Quiñones-Grueiro M, Provan G. A Novel Hybrid Approach for Fault-Tolerant Control of UAVs based on Robust Reinforcement Learning[C]//2021 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2021: 10719-10725.

[23] F. Fei, Z. Tu, D. Xu and X. Deng, "Learn-to-Recover: Retrofitting UAVs with Reinforcement Learning-Assisted Flight Control Under Cyber-Physical Attacks," 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 2020, pp. 7358-7364, doi: 10.1109/ICRA40945.2020.9196611.

[24] N. Bernini, M. Bessa, R. Delmas, A. Gold, E. Goubault, R. Pennec, S. Putot, and F. Sillion, "A few lessons learned in reinforcement learning for quadcopter attitude control," in Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control, 2021, pp. 1–11.

[25] Nguyen N P, Hong S K. Fault diagnosis and fault-tolerant control scheme for quadcopter UAVs with a total loss of actuator[J]. Energies, 2019, 12(6): 1139.